Taaltheorie en Taalverwerking

BSc KI / Artificial Intelligence

2026 – Lecture 3

Katrin Schulz

k.schulz@uva.nl

Institute for Logic, Language, and Computation

University of Amsterdam

Last time: recap

- Morphology deals with the processes governing the internal structure of words
- Syntax deals with the internal structure of sentences
  - Semantics is a separate topic! *Colourless green ideas sleep furiously* has no syntax errors.
- Grammars allow us to define well-formed sentences of a language and to assign internal structure to them
- Within a grammar for a natural language, we distinguish between the lexicon (*cat*, *of*, *the*) and the phrase structure rules ($S \rightarrow NP\ VP$, $PP \rightarrow P\ NP$)

Formal grammars

A grammar can be specified by four parameters:

- Σ a finite alphabet of terminal symbols
- N a finite set of non-terminal symbols
- S a special symbol $S \in N$ called the start symbol
- R a set of rules or productions consisting of
  - a sequence of (non-)terminal symbols α,
  - the symbol '$\rightarrow$'
  - another sequence of (non-)terminal symbols β

$$\alpha \rightarrow \beta, \text{ where } \alpha, \beta \in (N \cup \Sigma)^*$$

- Like regular expressions and FSAs, grammars can be used to define formal languages (sets of strings)
- Formal grammars can be used to define languages that cannot be captured by FSAs (while regular grammars are equivalent to FSAs and regexes!)

Today

1. Derivations with formal grammars
2. Regular languages and the Pumping Lemma
   - proving complexity statements
3. Chomsky's hierarchy
   - formalizing language complexity
4. Feature grammars
   - avoiding combinatorial rule explosions
5. CFGs and Feature Grammars in NLTK

Derivations

A formal grammar:

- Σ a finite alphabet of terminal symbols
- N a finite set of non-terminal symbols
- S a special symbol $S \in N$ called the start symbol
- R a set of rules or productions $\alpha \rightarrow \beta$, with $\alpha, \beta \in (N \cup \Sigma)^*$

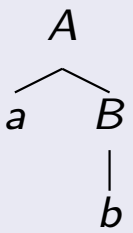
- The language defined by a grammar is *the set of strings composed of terminal symbols* that can be derived from the grammar's rules.
- Each sequence of rules that produces a string of the language is called a derivation.

Derivations

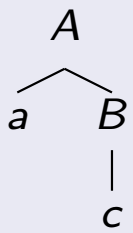
Grammar:
$N = \{A, B\}$
$\Sigma = \{a, b, c\}$
start symbol: A

1. $A \rightarrow a\ B$
2. $B \rightarrow b$
3. $B \rightarrow c$

Derivation 1:
First we apply
rule 1
then rule 2



Derivation 2:
First we apply
rule 1
then rule 3



The language defined by the grammar therefore is $\{ab, ac\}$.

- The first rule to be applied must begin with the start symbol.
- To apply a rule, we 'rewrite' the left symbol with the right sequence.
- The derivation finishes when we end up with terminal symbols.
- The resulting string of terminal symbols is a string in the language defined by the grammar.

Grammars for regular languages

Regular expressions/ FSAs are equivalent to *regular grammars*

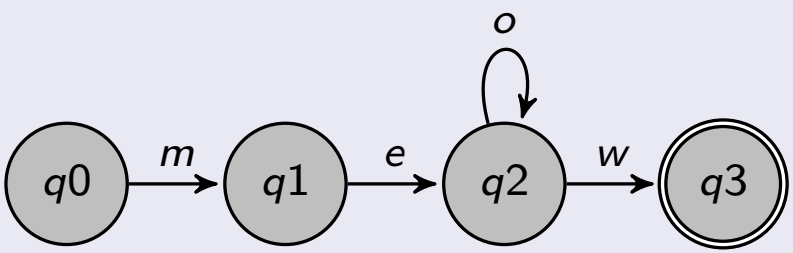
- For any regular expression or FSA, we can design an equivalent grammar:
  - Terminals are the input symbols.
  - Non-terminals are the states.
  - For every transition $(\langle X, a\rangle, Y)$, we have a production $X \to aY$.
  - For every accepting state X, we have a production $X \to \epsilon$.
- This kind of regular grammar is called a right-linear grammar
- Regular grammars are either left-linear or right-linear
  - What do you think is a left-linear grammar?
  - $X \to Ya$, we have a production $X \to Ya$

Grammars for regular languages

Regular expressions/ FSAs are equivalent to *regular grammars*

- For any regular expression or FSA, we can design an equivalent grammar:
  - Terminals are the input symbols.
  - Non-terminals are the states.
  - For every transition $X \rightarrow aY$, we have a production $X \rightarrow aY$.
  - For every accepting state X, we have a production $X \rightarrow \epsilon$.

$me(o)^*w$



Option 1:
$Q_0 \rightarrow m\ Q_1$
$Q_1 \rightarrow e\ Q_2$
$Q_2 \rightarrow o\ Q_2$
$Q_2 \rightarrow w\ Q_3$
$Q_3 \rightarrow \epsilon$

Option 2:
$Q_0 \rightarrow m\ Q_1$
$Q_1 \rightarrow e\ Q_2$
$Q_2 \rightarrow o\ Q_2$
$Q_2 \rightarrow w$

Exercise: regular expressions for grammars

$N = \{A, B\}$ $\Sigma = \{a, b, c\}$ start symbol $= A$ $A \rightarrow a\ B$ $B \rightarrow b$ $B \rightarrow c$	$N = \{NP, N, PP\}$ $\Sigma = \{cat, hat, the, in\}$ start symbol: NP $NP \rightarrow the\ N$ $NP \rightarrow the\ cat\ PP$ $NP \rightarrow the\ hat\ PP$ $PP \rightarrow in\ NP$ $N \rightarrow hat$ $N \rightarrow cat$

Can we find regular expressions that are equivalent to (i.e., define the same language as) these example grammars?

???	**???**

Exercise: regular expressions for grammars

1. Go to wooclap.com
2. Enter the event code in the top banner

Event code
XAYUMJ

Enable answers by SMS

Exercise: regular expressions for grammars

$N = \{A, B\}$ $\Sigma = \{a, b, c\}$ start symbol $= A$	$N = \{NP, N, PP\}$ $\Sigma = \{cat, hat, the, in\}$ start symbol: NP
$A \rightarrow a\ B$ $B \rightarrow b$ $B \rightarrow c$	$NP \rightarrow the\ N$ $NP \rightarrow the\ cat\ PP$ $NP \rightarrow the\ hat\ PP$ $PP \rightarrow in\ NP$ $N \rightarrow hat$ $N \rightarrow cat$

Can we find regular expressions that are equivalent to (i.e., define the same language as) these example grammars?

$a(b\|c)$	$the\ (cat\|hat)\ (in\ the\ (cat\|hat))^*$

Regular languages

Regular grammars (a type of formal grammars, subset of context-free grammars) are a completely equivalent alternative to FSAs and regular expressions for describing regular languages

Figure 2.9 Finite automata, regular expressions, and regular grammars are all equivalent ways of describing regular languages.

Not all formal languages are regular

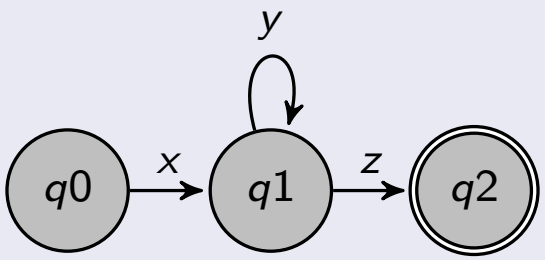
Can we use regular expressions/FSAs/regular grammars to define any formal language?

- For *finite* languages? – yes (finite languages are regular!)
- How about *infinite* languages L? – some

Not all formal languages are regular

How about *infinite* languages L? – some

- To be regular, L must be recognised by some FSA, which by definition has a finite number of states.
- Since L is infinite, it must include strings that are longer than the number of states in the FSA → the FSA must contain a loop.
- If x is the substring up to the loop, y the substring on the loop, and z be the substring following the loop ending on a final state.



- Regardless of how many times we traverse the loop, the resulting string will be part of the language: all strings xy^nz for $n \geq 0$.

The pumping lemma

All these observations are summarized in the Pumping Lemma, so called because the substring y is said to be 'pumped'.

Pumping Lemma

Let L be an infinite **regular** language. Then the following – theoretical – 'pumping' condition applies:

- There is a string $xyz \in L$ such that $y \neq \epsilon$ and any ('pumped') string $xy^n z$ for $n \geq 0$ also belongs to L.

In practice, if the pumping condition does not hold (if there is no string $xyz\ldots$), then L is not regular.

- The Pumping Lemma can be used to prove that a language is *not* regular: if we can show that there are no strings in L for which the pumping condition holds, then the language is not regular.
- In contrast, showing that L does satisfy the Pumping Lemma (there is a string $xyz\ldots$), is insufficient to prove that L is regular.

Using the pumping lemma: sketch

Consider $L = \{a^n b^n \mid n \geq 0\}$.

We can use the Pumping Lemma to show that L is not regular.

Stel L is a regular language. Then, with the pumping lemma, it follows:

- Some string xyz with $y \neq \epsilon$ is part of L such that $xy^n z \in L$ for all $n \geq 0$.
- Since y in xyz must be non-empty, y can only be either:
  1. some number of a's, or
  2. some number of b's, or
  3. some a's followed by some b's.
- Spoiler: If we pump y in any of the three cases above, we obtain a string that does not belong to L. Thus, it is not the case that $xy^n z \in L$ for all $n \geq 0$.

Hence, L cannot be regular.

Using the pumping lemma: full example

Consider $L = \{a^n b^n \mid n \geq 0\}$. If L is a regular language, then the pumping condition must hold: there must be a string $xyz \in L$ ($y \neq \epsilon$) such that any string $xy^m z$ (for $m \geq 0$) also belongs to L.

Since y must be non-empty, y can only be one of three options:

1. Some number $i > 0$ of a's, in which case: $x = a^{n-i}, y = a^i, z = b^n$.
   The string $a^{n-i}(a^i)^m b^n$ does for any $m \geq 0$ not belong to L: *Let $m = 3$, the number of a's $[(n-i) + i * 3]$ is higher than the number of b's (n).*
2. Some number $i > 0$ of b's, in which case: $x = a^n, y = b^i, z = b^{n-i}$.
   The string $a^n(b^i)^m b^{n-1}$ does not for any $m \geq 0$ belong to L: *Let $m = 3$, the number of b's $[(n-i) + i * 3]$ is higher than the number of a's (n).*
3. Some number $i > 0$ of a's followed by some number $j > 0$ of b's, in which case: $x = a^{n-i}, y = a^i b^j, z = b^{n-j}$.
   The string $a^{n-i}(a^i b^j)^m b^{n-j}$ for any $m \geq$ does not belong to L. *Let $m = 3$; we get $a^{n-i} a^i b^j a^i b^j a^i b^j b^{n-j}$ which is not in $a^n b^n$.*

We have found an m for which the pumping condition does not hold; L is not regular.

Using the pumping lemma: remarks

- The Pumping Lemma can only be used to show that an infinite language L is *not* regular.
  - If it is not possible to find a string $xyz \in L$ ($y \neq \epsilon$) such that any string $xy^m z$ (for any $m \geq 0$) also belongs to L, then we can conclude that L is not regular.
  - But if it is possible to find such a string, we cannot conclude that L is regular (a non-regular language may include such a string).
  - The easiest way to show that a language *is* regular, is to give a **regular expression, FSA, or right-linear grammar** for it.
- In the previous slide, we have shown that for $L = \{a^n b^n \mid n \geq 0\}$ it is not possible to find such a string (we have shown this by going over all logical possibilities). Hence we can conclude that L is not regular.

Limitations of finite automata: intuition

$$L = \{a^n b^n \mid n \geq 0\}$$

- An automaton scanning a string of the form $a^* b^*$ would have to *remember* how many a's it has seen, for arbitrarily long strings of a's and b's.
- You cannot distinguish between infinitely many different cases with only finite states.
- Languages that require an ability to count are not regular
  - the language $\{a^n b^n \mid n \geq 0\}$
  - the language of well-matched sequences of brackets
    - A sequence x is well-matched if it contains the same number of opening brackets '(' and closing brackets ')', and no initial subsequence y of x contains more closing than opening brackets.

Beyond regular languages

- The language $L = \{a^n b^n \mid n \geq 0\}$ is not regular.
- But can we specify it with a grammar? Yes:

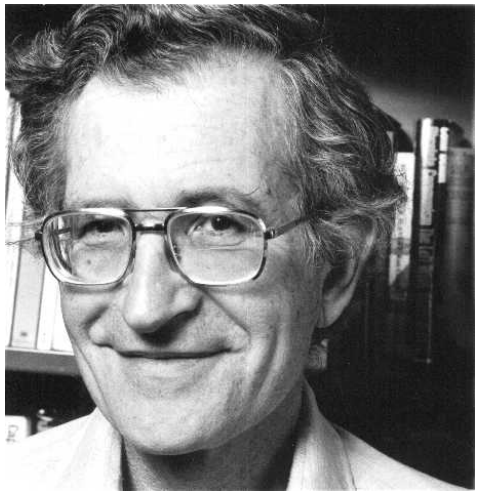
$$S \rightarrow a\ S\ b$$
$$S \rightarrow \epsilon$$

- Regular expressions, FSAs, and linear grammars can only describe a limited set of formal languages: the so-called regular languages.
- Every regular expression or FSA has an equivalent formal grammar, *but the reverse does not hold:*
  - Not all formal grammars are equivalent to a regex/FSA; only right-linear (and left-linear) grammars are.
  - some type of grammars can describe a bigger set of languages.

When a formalism or grammar can define a formal language that another one cannot define, we say that it has greater generative power or greater complexity.

The Chomsky Hierarchy

Noam Chomsky (1959):
"On certain formal properties of grammars",
Information and Control, 2(2):137–167.



Chomsky proposed to classify formal grammars into 4 types, which differ in their generative capacity – in the complexity of the languages they are able to generate or recognise.

Types of grammars in the Chomsky hierarchy

The different types of grammars within the hierarchy differ with respect to the types of grammar rules they require.

Let A and B be non-terminal symbols, x be an arbitrary string of terminal symbols, and α, β, and γ be arbitrary strings of terminal and non-terminal symbols.

Type 0 Recursively enumerable or unrestricted grammars.
Rule skeleton: $\alpha \rightarrow \beta \ (\alpha \neq \epsilon)$

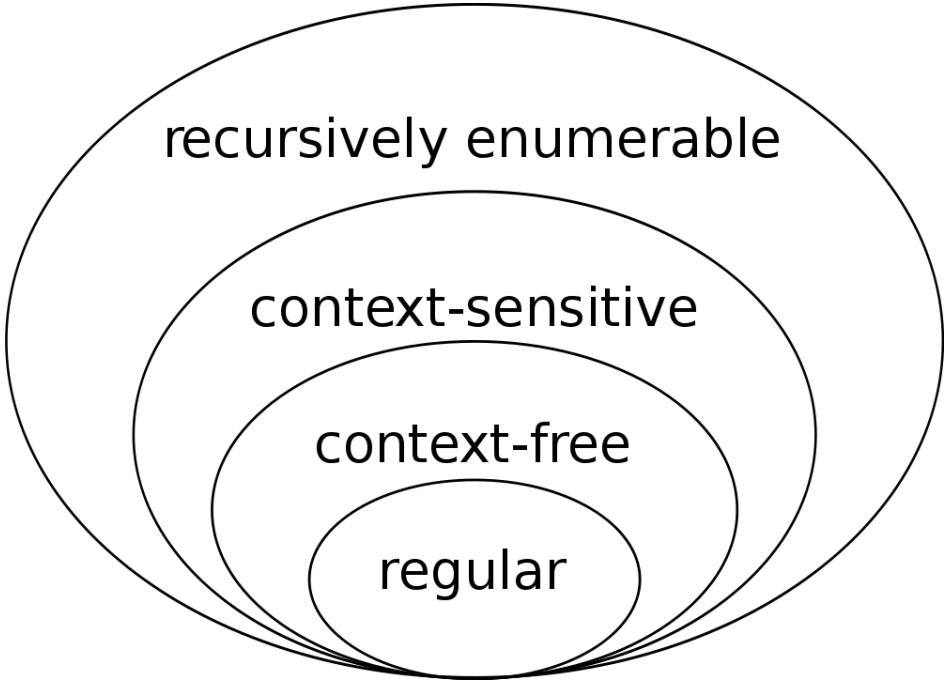
Type 1 Context-sensitive grammars.
Rule skeleton: $\alpha A \beta \rightarrow \alpha \gamma \beta \ (\gamma \neq \epsilon)$

Type 2 Context-free grammars.
Rule skeleton: $A \rightarrow \gamma$

Type 3 Regular grammars or right-linear grammars.
Rule skeleton: $A \rightarrow x \ B$ or $A \rightarrow x$

The Chomsky Hierarchy

This classification acts as a subsumption hierarchy: the set of languages described by grammars of greater power subsumes the set of languages described by grammars of less power (see 16.1).



There is a balance between expressibility and computational efficiency: grammars with *less* expressive power are computationally *more* tractable.

Grammars for natural languages

We have seen that natural language morphology can be defined by FSAs (regular). What about natural language syntax?

Center embeddings

(1) The cat ran away.
(2) The cat that the dog chased ran away.
(3) The cat that the dog that the flea bit chased ran away.

Center embeddings

(1) The cat died.
(2) [[The cat]$_1$ [that [the dog]$_2$ [chased]$_2$] [ran away]$_1$].
(3) [[The cat]$_1$ [that [the dog]$_2$ [that [the flea]$_3$ [bit]$_3$] [chased]$_2$] [ran away]$_1$].

- These sentences are of the following form (isomporphic to $a^n b^n$), which we have shown is not regular:

$$\textit{the N (that the N)}^n\ \textit{V}^n\ \textit{died}$$

- These sentences are hard to process when n goes up (limited working memory), but there is no fixed limit of n for speakers

Are natural languages regular?

We have seen that some sentences in English are not regular:

$$L = \textit{the Noun } (\textit{that the Noun})^n \ (\textit{Verb})^n \ \textit{Verb}$$

But what about English as a whole?

Regular languages are closed under intersection. That is, if A and B are regular languages then $A \cap B$ is also a regular language.

- $A = \textit{the Noun } (\textit{that the Noun})^* \ (\textit{Verb})^* \ \textit{Verb}$ is a regular language.
- $B = \textit{English}$
- $L = \textit{the Noun } (\textit{that the Noun})^n \ (\textit{Verb})^n \ \textit{Verb}$ (can be seen as the intersection of A and English, $A \cap B$)
- If English were regular, then L would also be regular.
- Since L is not regular, we conclude that English is not regular.

Exercise: context-free grammars

1. Go to wooclap.com
2. Enter the event code in the top banner

Event code
XAYUMJ

Enable answers by SMS

Is natural language syntax context-free?

(1) The cat ran away.
(2) The cat that the dog chased ran away.
...

We can certainly define a CFG for our non-regular example above.

There has been a lot of discussion regarding whether all natural languages are context-free or not.

- Many languages, including English, seem to be context-free...
- ...but some languages, like Swiss German (and Dutch), have been proven not to be.

Is natural language syntax context-free?

There is another version of the Pumping Lemma that can used to show that a language is *not* context-free.

- Since it is rather mathematically complicated, we will not look into the details of this here.
- The lemma would allow us to show that the following language is not context-free:

$$a^n b^m c^n d^m$$

- The key feature that makes this language non context-free are the cross-serial dependencies it exhibits (as opposed to center-embedding dependencies).

Is natural language syntax context-free?

It has been shown that Swiss German includes constructions that are equivalent to the non-context-free formal language $a^n b^m c^n d^m$

... das mer em Hans es huus hälfed aastriiche

... that we Hans_{Dat} house_{Acc} helped paint

'... that we helped Hans paint the house'

... das mer d'chind em Hans es huus lönd hälfe aastriiche

... that we the children_{Acc} Hans_{Dat} house_{Acc} let help paint

'... that we let the children help Hans paint the house'

- The number of accusative NPs must equal the number of verbs selecting for an accusative.
- The number of dative NPs must equal the number of verbs selecting for a dative object.
- The order of NPs and verbs must be the same.

Shieber (1985). Evidence against the context-freeness of natural language. *Linguistics and Philosophy*, 8:333–343.

Is natural language syntax context-free?

This is similar in Dutch:

... dat Jan Piet de kinderen zag helpen zwemmen

... that Jan Piet the children saw help swim

'... that Jan saw Piet help the children swim'

... dat Jan Piet Marie de kinderen zag helpen leren zwemmen

... that Jan Piet Marie the children saw help teach swim

'... that Jan saw Piet help Marie teach the children to swim'

- In this case, the cross-serial dependencies are not visible at the level of the strings (because there is no case marking)
- From a formal language theory perspective, these Dutch constructions are eequivalent to $a^n b^n$, which is context-free
- But: there is a string dependency that arises from the requirement of subject-verb agreement. Thus, in some cases, these constructions may not be context-free after all.

Bresnan, Kaplan, Peters, and Zaenen (1982). Cross-serial dependencies in Dutch. *Linguistic Inquiry*, 13(4):613–635.

Manaster-Ramer (1987). Dutch as a formal language. *Linguistics and Philosophy*, 10(2):221–246

Summary

- Grammars are formalisms to define formal languages.
- The Chomsky hierarchy classifies grammars according to their generative capacity.
  - Grammars can be distinguished (hierarchically) by the type of rules they allow.
  - Those with more expressive power are computationally more costly.
- The Pumping Lemma can be used to show that a particular language does *not* belong to a level in the hierarchy.
  - We have only seen the Pumping Lemma for regular languages.
- The syntax of natural languages is not regular, and in some cases it does not seem to be context-free either.
- However, many aspects of natural language syntax can be modelled with CFGs or *mildly context-sensitive grammars*.

CFGs and agreement

How can we deal with agreement dependencies in a CFG?

$S \rightarrow NP\ \ VP$
$NP \rightarrow D\ \ N$
$NP \rightarrow Pro$
$VP \rightarrow V\ \ NP$
$D \rightarrow this \mid these$
$N \rightarrow student \mid students$
$Pro \rightarrow she \mid her$
$V \rightarrow greet \mid greets$

Some options:

$NPsg \rightarrow Dsg\ \ Nsg$
$NPpl \rightarrow Dpl\ \ NPp$
$Ssg \rightarrow NPsg\ \ VPsg$
$Spl \rightarrow NPpl\ \ VPpl$

With plain CFGs, we can only deal with this issue by adding extra rules, but this leads to an exploding number of grammar rules.

Instead: feature structures (a.k.a. attribute–value or unification grammars).

Feature structures

Replace atomic categories (NP-1p-sg) with feature structures.

$$\begin{bmatrix} \textbf{cat} & \text{NP} \\ \textbf{pers} & 3 \\ \textbf{num} & \text{sg} \\ \textbf{gend} & \text{masc} \end{bmatrix} \qquad \begin{bmatrix} \textbf{cat} & \text{VP} \\ \textbf{pers} & 3 \\ \textbf{num} & \text{sg} \\ \textbf{vform} & \text{finite} \end{bmatrix}$$

- A feature structure is a list of features (attributes) and their values.
- They are also called attribute–value matrices.
- Usually values are typed (to avoid, e.g. **num**: masc)

Unification of features structures

- Two feature structures A and B unify $A \sqcup B$ if they can be merged into one consistent feature structure C.

$$\begin{bmatrix} \textbf{cat} & \text{NP} \\ \textbf{num} & \text{sg} \\ \textbf{gend} & \text{masc} \end{bmatrix} \sqcup \begin{bmatrix} \textbf{cat} & \text{NP} \\ \textbf{pers} & 3 \end{bmatrix} = \begin{bmatrix} \textbf{cat} & \text{NP} \\ \textbf{num} & \text{sg} \\ \textbf{gend} & \text{masc} \\ \textbf{pers} & 3 \end{bmatrix}$$

- Else, unification fails.

$$\begin{bmatrix} \textbf{cat} & \text{NP} \\ \textbf{num} & \text{sg} \\ \textbf{gend} & \text{masc} \end{bmatrix} \sqcup \begin{bmatrix} \textbf{cat} & \text{NP} \\ \textbf{num} & \text{pl} \end{bmatrix} = \phi$$

CFGs and feature structures in NLTK

NLTK makes it very easy to write CFGs, including CFGs with feature structures!

- Take a look at the Jupyter notebook on Canvas!